Europolis

Etapa 1 + Etapa 2 + Etapa 3 / Actualizări curente (API & Implementare)

Larisa Condrache Alexandra Calotă Amelia Corici
Alexandra Ciuclea

Mai 2026

Cuprins

1 Contextul proiectului **4**

2 Arhitectura sistemului **5**

3 Modelul problemei **6**
3.1 Entități principale 6
3.2 Relații cheie 6
3.3 Fluxul general al unui joc 6

4 Actorii sistemului și Use-Cases **8**
4.1 Diagrama 8
4.2 Actorii sistemului 9
4.3 Use-Cases (rezumat) 9

5 Reguli de business **11**

6 Coduri de răspuns și modele de eroare **12**
6.1 Coduri HTTP utilizate 12
6.2 Model `ErrorResponse` 12

7 Descrierea operațiilor principale (implementate) **13**
7.1 Games 13
7.2 Players 13
7.3 Turns 13
7.4 Actions 13
7.5 Properties 14
7.6 Trades 14
7.7 Chat 14

8 Implementare (Etapa 2) **15**
8.1 Tehnologii folosite 15
8.2 Structura proiectului 15
8.3 Baza de date 15
8.4 Conexiunea PDO 16
8.5 Endpoint-uri implementate (Etapa 2) 17
8.6 Cum se pornește aplicația local 17
8.7 Testare API (exemple) 17
8.8 Logica de joc implementată 18
8.9 Interfața web actualizată 18
8.10 Actualizări Etapa 3 19
8.11 Detalii tehnice de implementare 19
8.11.1 Bootstrapping-ul aplicației 19
8.11.2 Conexiunea la baza de date 20

8.11.3 Structura logicii de joc 20
8.11.4 Reguli tehnice pentru chirii și START 20
8.11.5 Chat și persistență 20
8.11.6 Fluxul JSON 21
8.12 Etapa curentă și direcții de finalizare 21

9 Extinderi ulterioare **22**

1 Contextul proiectului

Jocul Europolis este un joc de societate în care 2–6 jucători concurează pentru acumularea de proprietăți și capital, cu scopul de a-i aduce pe ceilalți la faliment.

Actualizare Etapa 2: implementarea curentă include listarea jocurilor existente, vizualizarea jucătorilor cu ID, proprietăți cumpărate, ștergerea jocurilor/jucătorilor, vânzarea directă a proprietăților către bancă, upgrade pentru proprietăți și chat per joc.

Actualizări curente: au fost adăugate butonul de *dark mode*, un chat mai bine integrat în interfață, îmbunătățiri de UI pentru afișarea jucătorilor, refacerea window-urilor / modal-urilor și corecții pentru chirii, inclusiv bonusul de la START.

Proiectul constă în modelarea acestui joc ca o aplicație web, în care logica de joc este expusă printr-un API REST, iar clienții interacționează cu serverul prin mesaje HTTP în format JSON.

Separarea logicii de joc (backend) de interfața vizuală (frontend) permite:

- mai mulți clienți să joace simultan din dispozitive diferite;
- validarea regulilor într-un singur loc (serverul);
- posibilitatea extinderii ulterioare (AI, spectatori, replay).

2 Arhitectura sistemului

Sistemul este compus din trei componente principale:

Componentă	Responsabilitate
Client	Interfața vizuală. Afișează tabla, trimite acțiunile jucătorului, redă starea curentă.
API Server	Nucleul aplicației. Expune endpoint-uri REST, validează acțiunile conform regulilor, actualizează și returnează starea.
Baza de date	Persistă starea jocurilor, profilurile jucătorilor și istoricul turelor.

Comunicarea este **sincronă prin REST** pentru acțiunile jucătorilor. Opțional, notificările în timp real pot fi adăugate prin **WebSocket** sau **polling** periodic.

3 Modelul problemei

3.1 Entități principale

Entitate	Descriere
`Game`	O sesiune de joc; are o stare (*waiting / active / finished*), jucători și o tură curentă.
`Player`	Un participant; are un sold, o poziție pe tablă, proprietăți deținute și un status (activ / falimentar).
`Board`	Tabla de joc; conține 40 de căsuțe aranjate circular.
`Tile`	O căsuță a tablei (proprietate, taxă, șansă, comunitate, închisoare, START etc.).
`Property`	Un tip de `Tile` cumpărabil; aparține unui grup de culoare și poate fi upgrade-at.
`Building`	Casă sau hotel construit pe o proprietate.
`Trade`	O tranzacție de negociere între doi jucători (proprietăți + bani).
`Card`	O carte de șansă sau comunitate; are un efect (mișcare, plată, colectare, trimitere la închisoare).
`Turn`	O tură a unui jucător; conține zarurile aruncate, pozițiile și acțiunile disponibile.
`Bank`	Entitate de sistem; gestionează plățile, clădirile și vânzarea proprietăților.

3.2 Relații cheie

- Un `Game` conține mai mulți `Player` și o singură `Board`.
- Un `Player` poate deține zero sau mai multe `Property`.
- O `Property` poate avea clădiri, iar nivelul de upgrade influențează chiria.
- Un `Game` are o succesiune de `Turn`-uri, fiecare aparținând unui singur `Player`.
- Un `Player` poate iniția trade-uri și poate participa la chat-ul jocului.

3.3 Fluxul general al unui joc

Faza 1 — Inițializare

1. Un jucător creează un joc (`POST /games`), configurând numărul de participanți.
2. Ceilalți jucători se alătură (`POST /games/{id}/players`).
3. Host-ul pornește jocul (`PATCH /games/{id}` cu `status: active`).

Faza 2 — Desfășurare

1. Jucătorul curent aruncă zarurile (`POST /games/{id}/turns`).
2. Serverul calculează noua poziție și determină acțiunea declanșată pe căsuța respectivă.
3. Jucătorul execută acțiunea disponibilă (`POST /games/{id}/turns/{n}/actions`).
4. Jucătorii pot iniția trade-uri (`POST /games/{id}/trades`) și pot trimite mesaje în chat.
5. Tura se încheie și trece la următorul jucător.

Faza 3 — Finalizare

1. Un jucător ajunge la sold negativ și declară faliment (`PATCH /players/{id}` cu `status: bankrupt`).
2. Activele sale sunt redistribuite conform regulilor.
3. Jocul se termină când rămâne un singur jucător activ.

4 Actorii sistemului și Use-Cases

4.1 Diagrama

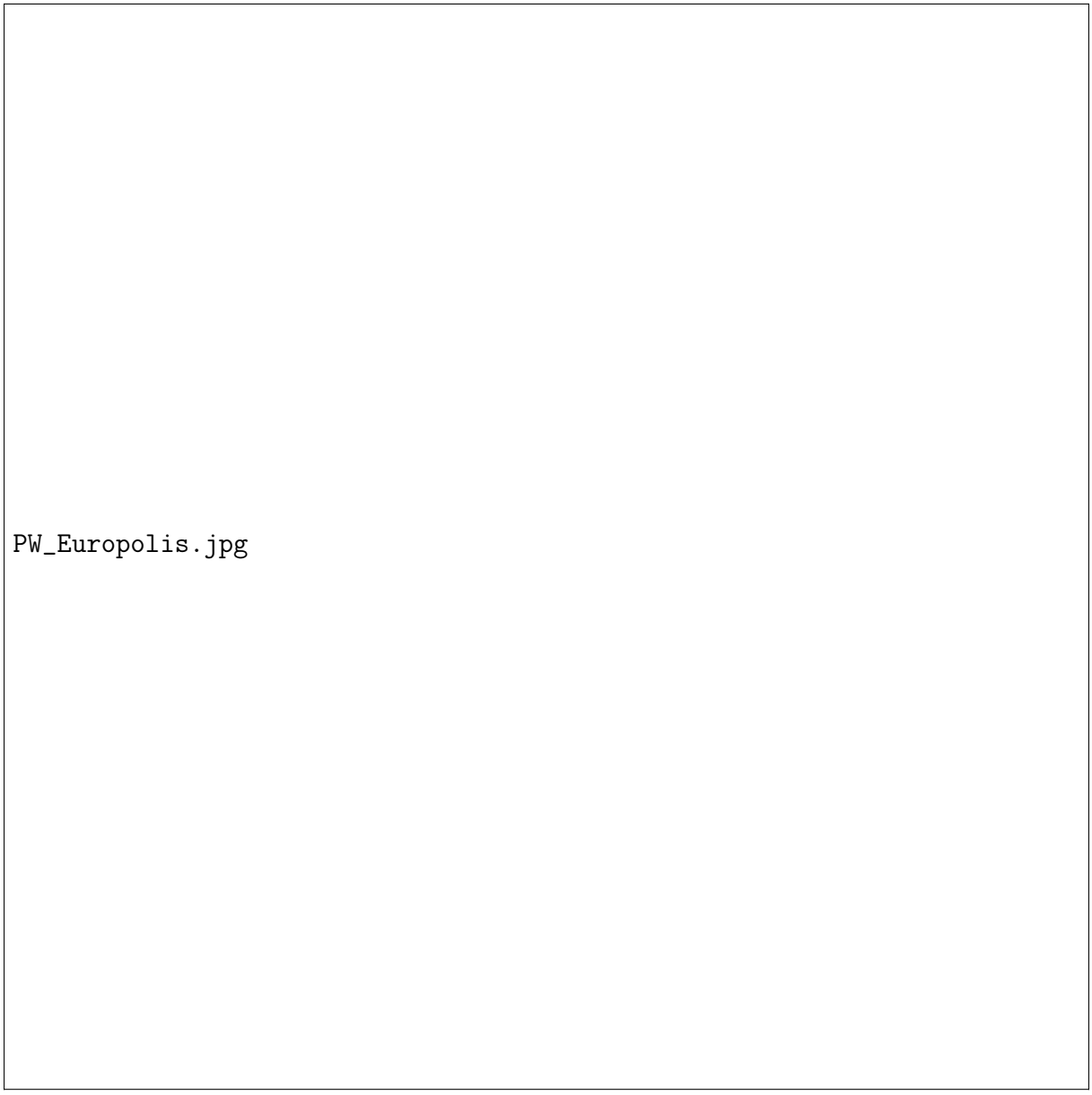


Figura 4.1: Diagrama Use-Case

4.2 Actorii sistemului

Actor	Rol
Jucător	Participare joc, executare acțiuni în tură proprie, inițiere trade-uri și chat.
Host / Creator joc	Inițiere joc, configurare, pornire, expulzare jucători și ștergere joc.
Bancă (sistem)	Gestionare automată tranzacții, validare reguli, distribuire cărți, acordare și colectare plăți.
Spectator *(opțional)*	Vizualizare stare joc, fără acțiuni de scriere.

4.3 Use-Cases (rezumat)

UC1 – Creare joc

Actor principal: Host
Scop: Inițierea unei sesiuni noi de joc.
Postcondiție: Jocul există și este pregătit pentru alăturarea altor jucători.

UC2 – Alăturare jucător

Actor principal: Jucător
Scop: Intrarea într-o sesiune existentă.
Postcondiție: Jucătorul este participant activ în joc.

UC3 – Vizualizare stare joc

Actor principal: Jucător
Scop: Obținerea informațiilor actuale despre joc.

UC4 – Inițiere tură

Actor principal: Jucătorul curent
Scop: Începerea turei de joc și aruncarea zarurilor.

UC5 – Executare acțiune

Actor principal: Jucătorul curent
Scop: Executarea unei acțiuni permise în tură (cumpărare, taxă, chirie etc.).

UC6 – Propunere trade

Actor principal: Jucător
Scop: Inițierea unui schimb între doi jucători.

UC7 – Răspuns trade

Actor principal: Jucător (destinatar)
Scop: Acceptarea sau respingerea unei oferte.

UC8 – Declarare faliment

Actor principal: Host / Sistem
Scop: Marcarea unui jucător ca bankrupt.

UC9 – Vizualizare jocuri existente

Actor principal: Jucător / Host
Scop: Clientul afișează lista jocurilor existente, statusul fiecărui joc, numărul de jucători și ID-ul jucătorului curent.

UC10 – Vânzare proprietate către bancă

Actor principal: Jucător
Scop: Jucătorul vinde o proprietate deținută către bancă și primește jumătate din suma plătită inițial.

UC11 – Chat de joc

Actor principal: Jucător
Scop: Schimb rapid de mesaje în cadrul sesiunii.

UC12 – Upgrade proprietate

Actor principal: Jucător
Scop: Construirea de case / hotel și recalcularea chiriei în funcție de nivel.

5 Reguli de business

Serverul validează regulile înainte de a executa operațiile expuse prin API.

Regulă	**Cod HTTP**
Date de intrare incomplete / invalide	`400`
Joc / jucător / tură inexistent(ă)	`404`
Conflict de stare (jocul nu mai acceptă jucători, joc plin etc.)	`409`
Proprietate deja cumpărată / fonduri insuficiente / acțiune invalidă	`422` sau `409`
Vânzare proprietate către bancă: `200`, refund = `purchasePrice / 2`	`200`

Pentru implementarea curentă sunt validate în special constrângeri de intrare și existența resurselor, iar regulile avansate rămân extinderi pentru etapa următoare.

6 Coduri de răspuns și modele de eroare

6.1 Coduri HTTP utilizate

Cod	Semnificație
`200 OK`	Operația a fost procesată cu succes.
`201 Created`	O resursă nouă a fost creată.
`400 Bad Request`	Cererea conține date invalide sau incomplete.
`404 Not Found`	Resursa solicitată nu există.
`409 Conflict`	Există un conflict de stare.
`422 Unprocessable Entity`	Regula de business nu permite executarea operației.

6.2 Model `ErrorResponse`

Toate erorile returnează un obiect JSON standardizat:

```
{
  "error": "NOT_FOUND",
  "message": "Jocul nu a fost gasit."
}
```

7 Descrierea operațiilor principale (implementate)

Specificatia OpenAPI a fost actualizata la versiunea 0.3.0. Ea include atât operațiile implementate în Etapa 2, cât și operațiile planificate pentru sprintul următor, marcate în YAML cu `x-implementation-status: planned`. Operațiile implementate pot fi demonstrate prin clientul web și prin scriptul de scenariu REST.

7.1 Games

POST `/games` creează o sesiune nouă de joc. Host-ul este adăugat ca prim jucător și jocul pornește în status *waiting*.
GET `/games` listează jocurile existente, împreună cu statusul, numărul de jucători și jucătorul curent.
GET `/games/{gameId}` returnează starea curentă a jocului.
PATCH `/games/{gameId}` modifică starea jocului.
DELETE `/games/{gameId}` șterge un joc și datele asociate.

7.2 Players

POST `/games/{gameId}/players` adaugă un jucător în joc.
GET `/games/{gameId}/players` returnează lista jucătorilor din joc.
GET `/games/{gameId}/players/{playerId}` returnează detalii despre un jucător din joc.
GET `/players/{playerId}` returnează detalii despre jucător după ID.
PATCH `/players/{playerId}` actualizează statusul unui jucător (ex. bankrupt).
DELETE `/games/{gameId}/players/{playerId}` elimină un jucător din joc.

7.3 Turns

POST `/games/{gameId}/turns` inițiază o tură și aruncă zarurile.
GET `/games/{gameId}/turns` listează turele unui joc.
GET `/games/{gameId}/turns/{turnId}` returnează detaliile unei ture.
GET `/games/{gameId}/turns/{turnId}/actions` listează acțiunile executate într-o tură.

7.4 Actions

POST `/games/{gameId}/turns/{turnId}/actions` execută o acțiune de joc.
Acțiunile curente includ: `buy_property`, `pay_rent`, `pay_tax`, `pay_jail`, `draw_card`, `go_to_jail`, `end_turn`.

7.5 Properties

GET `/games/{gameId}/properties` listează toate proprietățile cumpărabile și permite filtrare după proprietar sau grup.
GET `/games/{gameId}/properties/{propertyId}` returnează detaliile unei proprietăți.
GET `/games/{gameId}/properties/{propertyId}/buildings` returnează starea clădirilor de pe o proprietate.
POST `/games/{gameId}/properties/{propertyId}/buildings` construiește o casă sau hotel pe o proprietate.
POST `/games/{gameId}/properties/{propertyId}/sell` vinde o proprietate către bancă.

7.6 Trades

POST `/games/{gameId}/trades` creează o propunere de schimb.
GET `/games/{gameId}/trades` listează trade-urile unui joc.
GET `/games/{gameId}/trades/{tradeId}` returnează detaliile unei propuneri.
PATCH `/games/{gameId}/trades/{tradeId}` acceptă sau respinge o propunere.

7.7 Chat

GET `/games/{gameId}/chat` întoarce mesajele asociate jocului.
POST `/games/{gameId}/chat` adaugă un mesaj nou în chat-ul jocului.

8 Implementare (Etapa 2)

8.1 Tehnologii folosite

Tehnologie	Rol
PHP $\geq$ 8.1	Limbajul de programare principal.
Slim Framework 4.x	Rutare HTTP și API REST.
Composer 2.x	Management de dependențe.
MySQL / MariaDB	Baza de date relațională.
PDO	Acces la baza de date prin prepared statements.

8.2 Structura proiectului

```
Europolis/
  public/
    index.php                <- Entry point
    css/style.css
    europolis.html
    api_doc/
      index.html
      openapi.yaml
  src/
    Routes/
      games.php
      players.php
      turns.php
      trades.php
      properties.php
      chat.php
    Support/
      board.php
    Models/
      Board.php
  europolis.sql
  scripts/
    demo-scenario.ps1
  README.md
```

8.3 Baza de date

Baza de date `europolis` conține tabelele principale:

Tabel	Rol
games	Sesiuni de joc (status, max_players, current_player_id).
players	Jucători per joc (sold, poziție, status).
turns	Ture (zaruri + poziții).
actions	Acțiuni executate în ture (details JSON).
trades	Propuneri de schimb (pending/accepted/rejected).
trade_properties	Proprietăți incluse într-un trade.
owned_properties	Proprietăți cumpărate într-un joc (game_id, player_id, property_tile_index, purchase_price, building_level).
chat_messages	Mesaje de chat pe joc.

8.4 Conexiunea PDO

Clasa de conexiune implementează un singleton pentru a reutiliza aceeași instanță PDO pe durata request-ului HTTP. Configurarea folosește:

- `mysql:host=127.0.0.1;dbname=europolis;charset=utf8mb4`;
- user: `root`;
- parolă: goală;
- `ATTR_ERRMODE = ERRMODE_EXCEPTION`;
- `ATTR_DEFAULT_FETCH_MODE = FETCH_ASSOC`.

8.5 Endpoint-uri implementate (Etapa 2)

Metodă	Rută	Cod succes
GET	/games	200
POST	/games	201
GET	/games/{gameId}	200
PATCH	/games/{gameId}	200
DELETE	/games/{gameId}	200
GET	/games/{gameId}/players	200
POST	/games/{gameId}/players	201
GET	/games/{gameId}/players/{playerId}	200
DELETE	/games/{gameId}/players/{playerId}	200
GET	/players/{playerId}	200
PATCH	/players/{playerId}	200
GET	/games/{gameId}/turns	200
POST	/games/{gameId}/turns	201
GET	/games/{gameId}/turns/{turnId}	200
GET	/games/{gameId}/turns/{turnId}/actions	200
POST	/games/{gameId}/turns/{turnId}/actions	200
GET	/games/{gameId}/properties	200
GET	/games/{gameId}/properties/{propertyId}	200
GET	/games/{gameId}/properties/{propertyId}/buildings	200
POST	/games/{gameId}/properties/{propertyId}/sell	200
POST	/games/{gameId}/properties/{propertyId}/buildings	200
GET	/games/{gameId}/trades	200
POST	/games/{gameId}/trades	201
GET	/games/{gameId}/trades/{tradeId}	200
PATCH	/games/{gameId}/trades/{tradeId}	200
GET	/games/{gameId}/chat	200
POST	/games/{gameId}/chat	201

8.6 Cum se pornește aplicația local

1. Pornește MySQL (XAMPP sau serviciul local).
2. Creează schema rulând fișierul `europolis.sql`.
3. Instalează dependențele: `composer install`.
4. Pornește serverul: `php -S localhost:8080 -t public`.

8.7 Testare API (exemple)

```
# 1) Creare joc
POST http://localhost:8080/games
Body: {"hostName":"Ana","maxPlayers":4}

# 2) Adaugare jucator
```

```
POST http://localhost:8080/games/1/players
Body: {"name":"Mihai"}

# 3) Activare joc
PATCH http://localhost:8080/games/1
Body: {"status":"active"}

# 4) Creare tura
POST http://localhost:8080/games/1/turns
Body: {"playerId":1}

# 5) Actiune in tura
POST http://localhost:8080/games/1/turns/1/actions
Body: {"actionType":"buy_property"}

# 6) Listare proprietati
GET http://localhost:8080/games/1/properties

# 7) Vanzare proprietate catre banca
POST http://localhost:8080/games/1/properties/11/sell
Body: {"playerId":1}
```

8.8 Logica de joc implementată

- tabla de joc conține 40 de căsuțe;
- poziția este calculată cu formula `(pozitie_curenta + zar1 + zar2) % 40`;
- trecerea prin START acordă bonus;
- căsuțele de tip *property* / *station* / *utility* / *tax* / *chance* / *community* / *jail* au acțiuni dedicate;
- proprietățile pot fi cumpărate, vândute și upgrade-ate;
- chiria este recalculată în funcție de nivelul clădirilor;
- timpul în închisoare și plata taxei de ieșire sunt tratate explicit;
- trade-urile transferă proprietăți și bani între jucători;
- chat-ul este persistat în baza de date și afișat în clientul web.

8.9 Interfața web actualizată

- lista de jocuri include filtre vizuale pentru *active* și *waiting*;
- există un modal de selecție a jucătorului local;
- chat-ul este accesibil printr-un dock separat și prin modal dedicat;

- lista proprietăților cumpărate oferă acțiuni directe pentru upgrade și vânzare;
- layout-ul a fost ajustat astfel încât panoul de proprietăți să rămână compact;
- ferestrele modale au fost refăcute astfel încât fluxul de joc să fie stabil pe desktop și mobil;
- tabla este desenată cu hotspot-uri interactive și token-uri suprapuse.

8.10 Actualizări Etapa 3

- a fost introdus butonul de *dark mode*, pentru schimbarea rapidă a temei UI;
- chat-ul a fost adăugat și integrat mai bine în fluxul de joc;
- afișarea jucătorilor a fost refăcută pentru un aspect mai clar și distinct;
- chiriile pot fi editate și sunt recalculate corect în funcție de upgrade;
- la trecerea prin START se oferă 400 la prima trecere, apoi 200 la trecerile următoare;
- window-urile lipsă au fost refăcute și readuse în interfață;
- panoul proprietăților cumpărate nu mai lărgește pagina;
- aspectul general al aplicației a fost rafinat pentru o prezentare mai curată.

8.11 Detalii tehnice de implementare

8.11.1 Bootstrapping-ul aplicației

Aplicația pornește din `public/index.php`, unde sunt încărcate dependențele Composer, configurat containerul DI și definită conexiunea PDO. Fluxul de pornire include:

- activarea `body parsing middleware` pentru request-uri JSON;
- configurarea rutării Slim;
- middleware CORS pentru `GET, POST, PUT, PATCH, DELETE, OPTIONS`;
- servirea documentației statice din `/docs/`;
- încărcarea fișierelor de rutare pentru `games`, `players`, `turns`, `trades`, `properties` și `chat`.

8.11.2 Conexiunea la baza de date

Conexiunea este definită prin PDO și este injectată în containerul Slim. Configurarea curentă folosește:

- host local `127.0.0.1`;
- baza de date `europolis`;
- charset `utf8mb4`;
- `ERRMODE_EXCEPTION` pentru tratarea erorilor;
- `FETCH_ASSOC` pentru rezultatele query-urilor.

8.11.3 Structura logicii de joc

`src/Support/board.php` conține tabla de joc și funcțiile utilitare folosite de rutare:

- lista celor 40 de căsuțe și tipurile lor;
- calculul dimensiunii tablei;
- indexul pentru închisoare;
- progresia de upgrade (*1 casă*, *2 case*, *3 case*, *hotel*);
- calculul chiriei în funcție de preț și nivelul de upgrade;
- grupurile de culori ale proprietăților;
- calculul costului de upgrade;
- extragerea sigură a unei căsuțe după index.

8.11.4 Reguli tehnice pentru chirii și START

- chiria curentă este calculată din prețul de bază și procentul de upgrade;
- proprietățile au nivel de upgrade stocat în coloana `building_level`;
- la START se oferă 400 la prima trecere și 200 la trecerile următoare;
- serverul returnează acțiuni disponibile precum `buy_property`, `pay_rent`, `pay_tax` sau `draw_card` în funcție de tile.

8.11.5 Chat și persistență

Funcționalitatea de chat este persistată în tabela `chat_messages`. Mesajele sunt validate și trunchiate la maximum 500 de caractere, iar la citire sunt ordonate cronologic pentru afișare în UI.

8.11.6 Fluxul JSON

Toate rutele returnează JSON și includ mesaje de eroare standardizate. În funcție de endpoint, răspunsurile pot conține:

- metadate despre joc (`id`, `status`, `currentPlayerId`);
- stare jucători (`balance`, `position`, `status`, `properties`);
- informații despre tură (`dice`, `fromPosition`, `toPosition`, `availableActions`);
- detalii despre proprietăți (`ownerId`, `upgradeLevel`, `rent`, `nextUpgradeCost`);
- rezultate de trade și chat.

8.12 Etapa curentă și direcții de finalizare

- urmează implementarea testelor automate pentru scenariile principale;
- se urmărește publicarea aplicației astfel încât să fie accesibilă complet online;
- lipsa unei metode de logare / autentificare rămâne o limitare cunoscută;
- în rest, fluxul principal de joc este funcțional și potrivit pentru demonstrare.

9 Extinderi ulterioare

OpenAPI-ul 0.3.0 funcționează și ca roadmap: operațiile nefinalizate sunt păstrate în specificație pentru a ghida implementarea din etapa următoare.

- implementare completă a regulilor (tura curentă, ownership, chirii reale, construcții, ipotecare);
- resursă dedicată pentru `properties` și `buildings`;
- teste automate pentru endpoint-uri și scenarii de joc;
- autentificare și conturi persistente de utilizator;
- modul spectator și notificări în timp real (WebSocket);
- salvarea și reluarea jocurilor.